# TCD3
# Test Automation & Continuous Delivery

**Deployment & Monitoring**

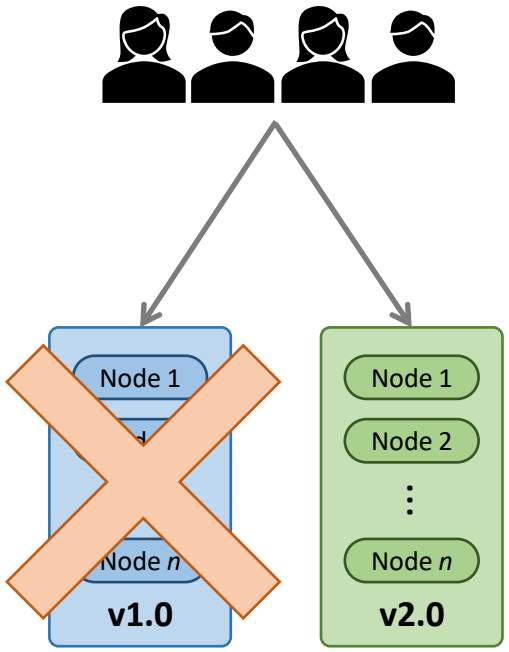
S. Wagner

# Deployment

- Goals
  - provide software to users (cf. agile manifesto)
  - do not negatively impact user experience of current users
  - if problems occur, roll back to a good version
- Manual vs. Automated Deployment
  - manual deployment is time consuming and error prone
  - stable and safe deployment process has to be automated
  - deployment should be integrated into CI/CD pipeline
- Deployment Strategies
  - define the general process for deploying a new version

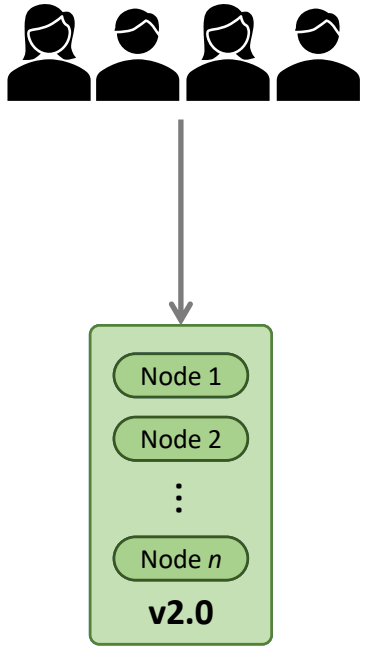
# Reckless Deployment

- aka Recreate, Basic, Simple
  - very simple and naïve strategy
  - negative effects on users are not considered
  - leads to an effective downtime
  - very easy to implement
- Steps:
  - decommission old version
  - deploy new version



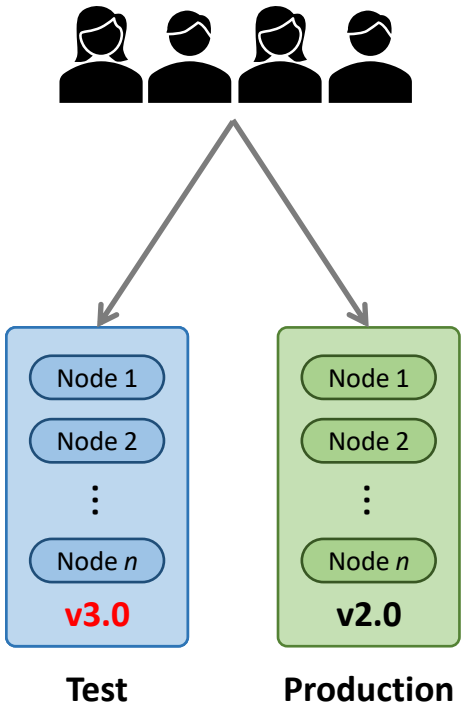
# Rolling Deployment

- aka Gradual, Ramped, Incremental
  - system is gradually transitioned into new version
  - system is heterogeneous during the transition phase
  - users use old and new version arbitrarily
  - backward compatibility is essential
  - no downtime, if no problems occur
- Steps:
  - replace one node with new version
  - monitor health of new node
  - repeat until all nodes are replaced



# Blue/Green Deployment

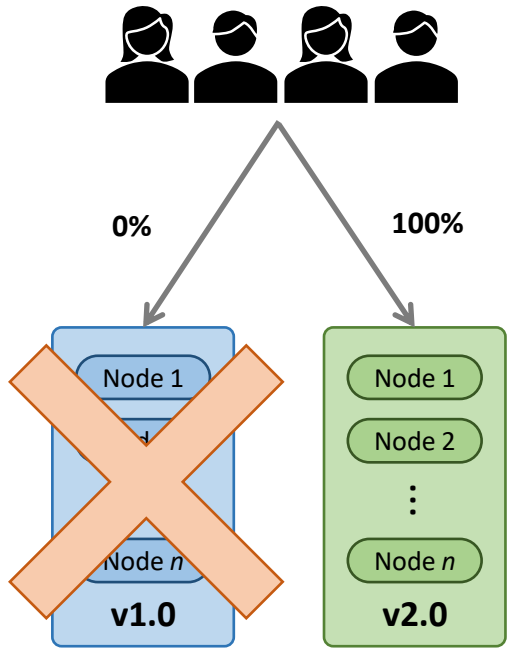
- aka Red/Black, Staged
  - infrastructure consists of two identical environments
  - one is used for production and one for testing
  - at deployment both environments are swapped
  - rather costly as infrastructure is duplicated
- Steps:
  - test new version in identical test environment
  - test environment becomes new production environment
  - old production environment becomes test environment for next version



# Canary Deployment

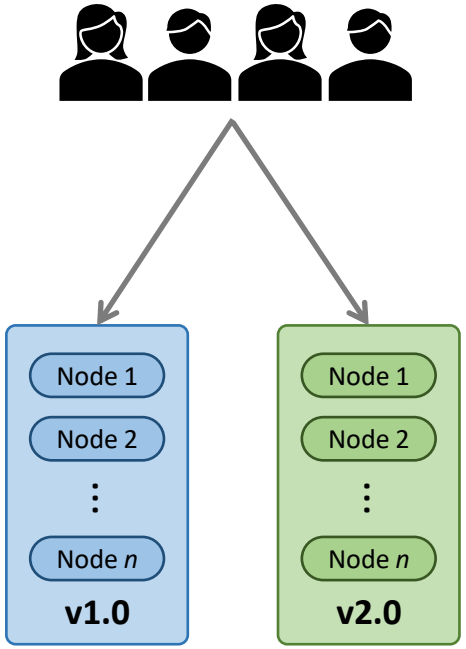
- aka Ringed, Silent Launch, A/B Testing
  - gradually increase the number of users who get the new version
  - monitor new version under real-life conditions
  - new version serves as a test instance
  - split users between old and new version by traffic percentage
  - other criteria can also be used for splitting (e.g., location, device, user group, etc.)

- Steps:
  - create new environment for new version
  - provide new version only to a small number of users
  - monitor how new version performs
  - gradually increase number of users
  - decommission old version, after all users have been shifted



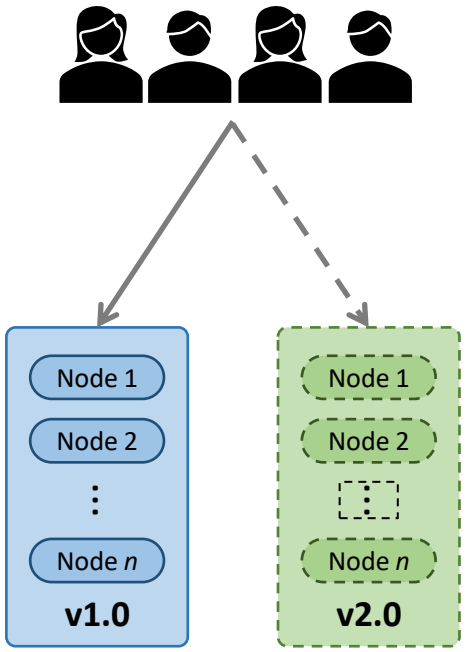
# Versioned Deployment

- Parallel Deployment of Different Versions
  - new version is offered side by side with old version
  - versions are distinguished by their routes
  - e.g., very useful for different API versions
- Steps:
  - deploy new version alongside of old version
  - adjust resource capacities as users shift to new version



# Shadow Deployment

- Shadow Environment for Testing
  - new version receives copy of real-world data
  - response from new version is ignored
  - enables testing with real-world data
  - infrastructure is duplicated
- Steps:
  - create new environment for new version
  - duplicate all traffic and route it to new version



# Deployment Strategies Summary

| Strategy | No Downtime | Real Traffic | Costs | Rollback Time | User Impact | Complexity |
|---|---|---|---|---|---|---|
| Reckless | ✗ | ✗ | ✓ | ✗ | ✗ | ✓ |
| Rolling | ✓ | ✗ | ✓ | ✗ | ✓ | ✓ |
| Blue/Green | ✓ | ✗ | ✗ | ✓ | ✓ | ✓ |
| Canary | ✓ | ✓ | ✓ | ✓ | ✓ | ~ |
| Versioned | ✓ | ✗ | ~ | ✓ | ✓ | ~ |
| Shadow | ✓ | ✓ | ✗ | ✓ | ✓ | ✗ |

# System Health Monitoring

- Health Monitoring is Essential for Automated Deployment
  - health of the system has to be checked continuously during/after deployment
  - any problems or errors must be detected and reported as early as possible
  - if the new version does not operate as expected, it has to be rolled back
- Collect, Visualize, and Analyze Logs and Metrics
  - logs and metrics should be collected and aggregated at a single point
- Logs vs. Metrics
  - logs are textual messages created by the system during operation
  - metrics are numerical values which describe the system's performance
  - metrics can be compared with thresholds to trigger alerts

# Prometheus

https://prometheus.io

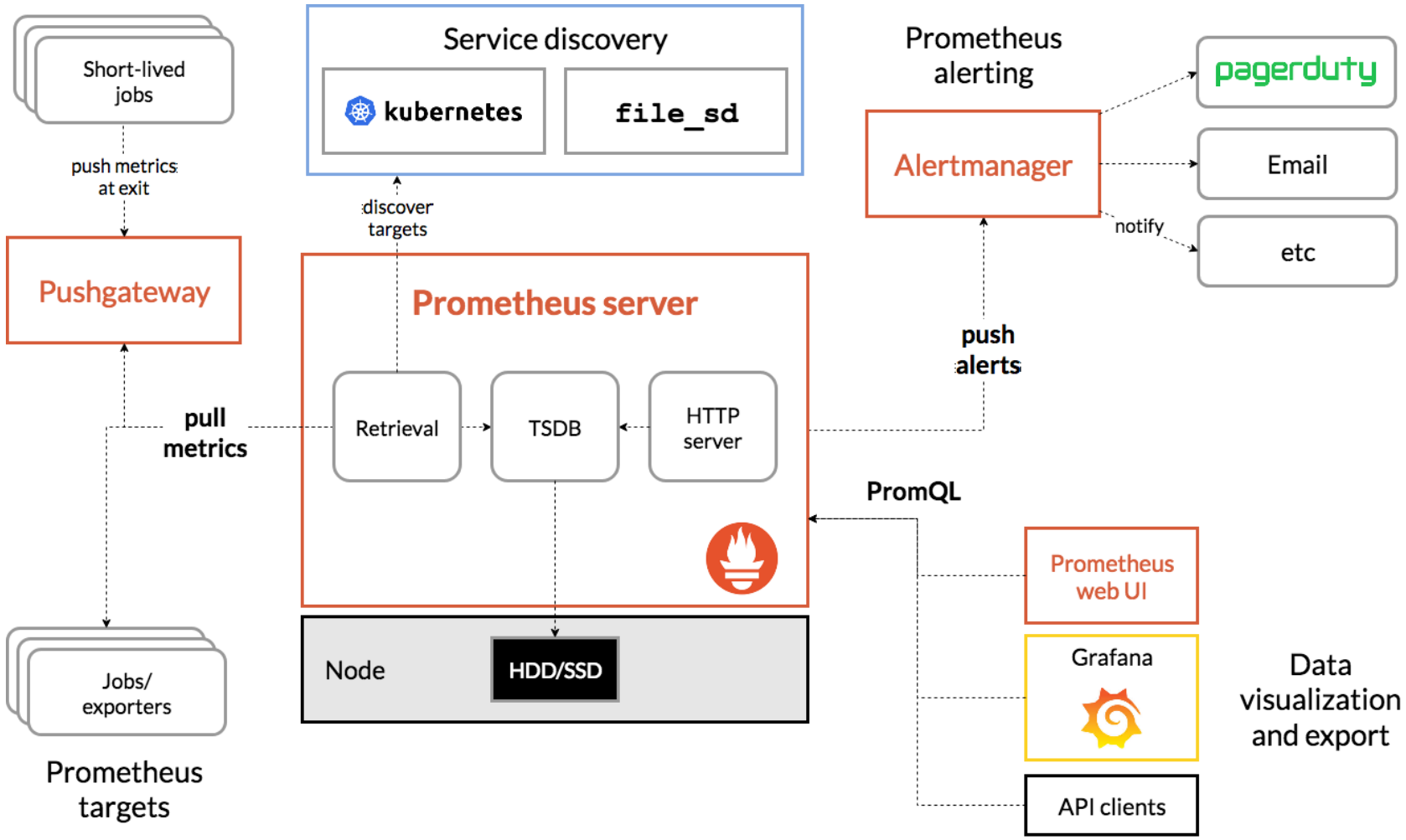



- What is Prometheus?
  - open-source systems monitoring and alerting toolkit
  - collects and stores metrics in a time series database

- Features
  - multi-dimensional data model with metrics and labels
  - PromQL query language
  - targets are statically configured or dynamically discovered
  - metrics are pulled over HTTP
  - intermediary gateway also supports pushing

# Prometheus

https://prometheus.io




https://prometheus.io/docs/introduction/overview

# Grafana

https://grafana.com




- What is Grafana?
  - open-source analytics and interactive visualization web application
  - provides dashboards for data retrieved from various data sources

- Features
  - custom dashboards and dashboard templates
  - lots of predefined dashboards and a very active community
  - support of many different data sources (e.g., Prometheus, InfluxDB, Elasticsearch, MySQL, MS SQL Server, MongoDB, Splunk, …)
  - authentication, permission management, and multi-tenancy
  - data transformations and annotations

# Grafana

https://grafana.com




https://play.grafana.org